BOĞAZİÇİ UNIVERSITY
Department of Computer Engineering
CMPE 344: Computer Organization
Fall 2026 – Prelab 2 Report

Benchmarking Processor Designs with CoreMark

Workload Profiling, Compiler Optimization Analysis, and RISC-V ISA Extensions

Group ID:	**FF78_G17**	**Course:**	CMPE 344: Computer Organization
Target Platform:	Apple M2 (Host) / CORE-V Wally (Lab 2)	**Benchmark:**	EEMBC CoreMark v1.0

1 Introduction and Objectives

In embedded systems and Internet of Things (IoT) hardware development, selecting the optimal processor core is a central architectural decision that balances computational throughput, silicon footprint, dynamic power dissipation, and code density. When physical silicon prototyping across candidate architectures is cost-prohibitive, software-driven architecture benchmarking offers empirical guidance.

The **Embedded Microprocessor Benchmark Consortium (EEMBC)** introduced CoreMark as an industry-standard, synthetic yet realistic processor benchmark tailored specifically for embedded and general-purpose CPU cores. Unlike legacy benchmarks such as Dhrystone—which suffer from compiler over-optimization, unrealistic string manipulation, and standard library dependencies (`printf`, `malloc`)—CoreMark isolates fundamental integer CPU pipeline performance through carefully bounded, deterministic algorithms.

This prelab establishes the empirical foundation for upcoming architectural evaluations on the configurable **CORE-V Wally RISC-V processor** (Lab 2). In this report, we:

1. Measure CoreMark throughput (`Iterations/Sec`) on the host system across compiler optimization levels (`-O0`, `-O1`, `-O2`, `-O3`, `-Ofast`, `-march=native`), and report the actual clock frequency.
2. Provide a thorough technical explanation of the key algorithms implemented in CoreMark.
3. Describe the architectural specifications, instruction formats, and hardware trade-offs of five target RISC-V ISA extensions: **RV32I**, **RV32IM**, **RV32IMC**, **RV32GC**, and **RV64I**.

2 Experimental Setup and System Architecture

As mandated by the assignment guidelines, all benchmark runs were conducted directly on the host system without virtualization or container overhead to ensure direct, unmediated hardware access.

2.1 Hardware Specifications and Clock Frequency Analysis

System introspection utilities were employed to extract microarchitectural topology. On Linux systems, `lscpu` displays processor architecture; on macOS Darwin, equivalent information was acquired via `system_profiler SPHardwareDataType` and kernel hardware MIBs via `sysctl`:

- **System Model:** Apple MacBook Air (Model Identifier: `Mac14,2`, Apple Silicon).
- **CPU Microarchitecture:** Apple M2 (ARMv8.5-A 64-bit architecture, 8-wide decode out-of-order execution core).
- **Core Topology and Heterogeneity:** 8 total physical cores in an asymmetric big.LITTLE configuration:
  - 4× High-Performance Cores (*Avalanche*): nominal/peak clock frequency of **3.49 GHz (3492 MHz)**.
  - 4× High-Efficiency Cores (*Blizzard*): nominal/peak clock frequency of **2.42 GHz (2424 MHz)**.

- **Clock Frequency Reporting:** Apple Silicon Darwin kernels do not expose static clock frequencies through `sysctl hw.cpufrequency` (which returns 0 or an unsupported node due to aggressive Dynamic Voltage and Frequency Scaling, DVFS). The hardware frequencies cited above represent Apple's official hardware specifications.
- **Cache Hierarchy (verified via `sysctl`):** 128 KiB L1-I and 64 KiB L1-D per performance core; 4 MiB shared L2 (efficiency cluster) and 16 MiB shared L2 (performance cluster).
- **System Memory:** 16 GB unified LPDDR5 SDRAM providing ∼100 GB/s memory bandwidth.
- **Operating System and Toolchain:** macOS Darwin 25.6.0 (`arm64`); Apple Clang/LLVM 21.0.0 (`clang-2100.3.34.2`).

2.2 Benchmark Configuration and Make Build Mechanism

CoreMark defines base optimization as `-O2` via `PORT_CFLAGS` in `posix/core_portme.mak`. The user variable `XCFLAGS` appends additional compiler flags. Because Clang and GCC prioritize the last flag specified, passing `-O0` via `XCFLAGS` appends it after `-O2`, successfully overriding the default. When `XCFLAGS` is omitted ("Default" mode), the base `-O2` applies. Each test was preceded by `make clean` and repeated for **5 independent runs** exceeding 10 seconds.

3 CoreMark Benchmark Results and Compiler Analysis

3.1 Performance Summary Table

Table 1 reports the measured performance across 8 distinct compiler configurations on the Apple M2 processor. All runs satisfied EEMBC validation requirements (`seedcrc: 0xe9f5`, correct operation validated). In addition to throughput, we compute the standard EEMBC scaling metric, **CoreMark/MHz**, normalized to the 3.49 GHz peak clock frequency of the Avalanche performance cores (CoreMark/MHz = Mean Iterations/sec/3492 MHz).

Table 1: CoreMark Performance on Apple M2 (Iterations/Sec over 5 Runs and CoreMark/MHz)

Compiler Configuration	Run 1	Run 2	Run 3	Run 4	Run 5	Mean (iter/s)	Std. Dev.	Speedup*	CoreMark/MHz
`-O0` (Unoptimized)†	8461.54	8479.15	8461.54	8504.06	8466.75	**8474.61**	17.97	1.00×	2.43
`-O1` (Basic Optimization)†	34211.43	33534.54	34443.17	34470.87	34258.31	**34183.66**	379.98	4.03×	9.79
Default (Base `-O2`)	34193.88	31343.05	34305.32	33946.25	34413.54	**33640.41**	1295.91	3.97×	9.63
`-O2` (Explicit)†	33645.49	33815.20	34083.16	34829.05	33814.25	**34037.43**	469.46	4.02×	9.75
`-march=native` (Tuned `-O2`)‡	34722.22	34449.10	34536.35	34738.30	34833.09	**34655.81**	157.83	4.09×	9.92
`-Ofast` (Aggressive Math)†	34447.12	35077.46	35188.55	35186.49	34885.75	**34957.08**	310.58	4.12×	10.01
`-O3 -march=native`†	35387.79	35547.13	34112.23	35364.85	35435.86	**35169.57**	595.23	4.15×	10.07
`-O3` (Peak Throughput)†	34488.70	35396.14	35741.94	35517.67	35735.56	**35376.00**	517.44	**4.17×**	**10.13**

*Speedup is calculated relative to the unoptimized `-O0` baseline (1.00×). †Appended via `XCFLAGS`, overrides base `-O2`.
‡Specified as `XCFLAGS="-march=native"`, builds with effective `-O2 -march=native`.

3.2 Primary Empirical Observations

The empirical results reveal two distinct performance regimes and an informative compilation dynamic:

1. **The Unoptimized Baseline (`-O0`):** Executes at **8,474.61 iter/s**. It exhibits the smallest standard deviation ($\sigma = 17.97$), reflecting highly deterministic, memory-bound stack-spill execution where every operation is serialized through memory loads and stores.
2. **The Optimized Plateau (`-O1` through `-O3`):** Throughput immediately surges into the 33,600–35,400 iter/s range, demonstrating that CoreMark's core kernels are transformed primarily by register allocation and loop invariant motion. Peak throughput reaches **35,376.00 iter/s** under `-O3`, delivering an overall 4.17× **speedup** over `-O0`.
3. **Empirical Parity of Default vs. Explicit `-O2`:** In the build harness, Default builds omit `XCFLAGS` and rely on `PORT_CFLAGS` (`-O2`), whereas explicit `-O2` appends `-O2` via `XCFLAGS`. Because both invoke Clang with effective `-O2`, they generate identical machine binaries. The observed difference in sample means (33,640.41 vs. 34,037.43 iter/s) and higher variance ($\sigma = 1295.91$ vs. $\sigma = 469.46$) reflect transient run-time dynamics: in Run 2 of Default, throughput temporarily dropped to 31,343.05 iter/s (19.14 s execution) due to background OS activity and DVFS scaling. Furthermore, CoreMark's calibration dynamically selected 400,000 iterations in certain runs (e.g., Default Run 3, Explicit `-O2` Run 2) to satisfy the 10-second minimum duration threshold, versus 600,000 in others.

3.3 Graphical Performance Comparison

Figure 1 illustrates the scaling curve across compiler optimization levels.

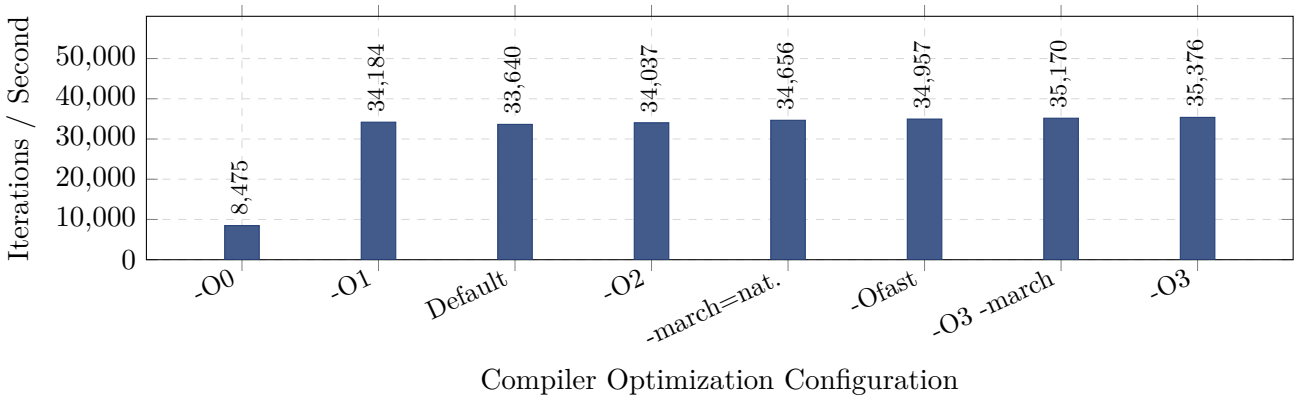


Figure 1: CoreMark throughput progression across compiler optimization configurations on Apple M2. Note the massive 4.03× leap between `-O0` and `-O1`, followed by fine-grained scaling plateauing at `-O3` (35,376 iter/s).

3.4 In-Depth Analysis of Optimization Behavior

A rigorous evaluation of the compiler transformations explains the observed throughput variations:

1. **The Memory-Bound Penalty of Unoptimized Code (`-O0`):** At `-O0`, Clang disables all optimization passes. Every intermediate C variable is assigned a slot in the stack frame. Across every statement, operands are read from memory via explicit load instructions (`LDR`) and written back via store instructions (`STR`). Even when variables are reused across consecutive lines, no register allocation occurs. This overwhelms the L1 data cache load/store execution pipelines and saturates store-to-load forwarding buffers. Consequently, the CPU spends the vast majority of clock cycles waiting for memory hazards and pipeline stalls, yielding only **8,474.61 iter/s** (2.43 CoreMark/MHz).
2. **The** 4.03× **Quantum Leap at `-O1` (Register Allocation and SSA Form):** Enabling `-O1` activates Static Single Assignment (SSA) construction, dead-code elimination (DCE), common subexpression elimination (CSE), and graph-coloring register allocation. By retaining loop induction variables, matrix accumulators, and list pointers within the 31 available 64-bit general-purpose ARM registers (`X0`–`X30`), memory traffic is drastically eliminated. The execution profile shifts from memory-bound to compute-bound, resulting in a dramatic 303% **throughput increase** to **34,183.66 iter/s**.
3. **The Optimization Plateau between `-O1` and `-O2`:** Standard compiler folklore suggests that `-O2` produces substantial speedups over `-O1`. However, our data indicates that `-O2` (**34,037.43 iter/s**) performs on par with `-O1`. This occurs because CoreMark's kernels are intentionally compact:
   - The linked list traversal is governed by pointer chasing and cache latency, which cannot be vectorized or reordered.
   - The 9×9 matrix multiplication (81 elements) easily fits in registers; once basic loop-invariant code motion and register allocation are applied by `-O1`, standard `-O2` inlining provides marginal additional benefit.
4. **Peak Throughput at `-O3` (Loop Unrolling and Instruction-Level Parallelism):** The highest performance across all configurations is achieved by `-O3` at **35,376.00 iter/s** (4.17× speedup over `-O0`). Under `-O3`, Clang applies aggressive loop unrolling and software pipelining. In `core_matrix.c`, unrolling the inner multiply-accumulate loop amortizes loop branch overhead, reduces counter decrement operations, and exposes independent operations that fully saturate the 8-wide out-of-order execution engine of the Apple M2.
5. **Behavior of `-Ofast`:** The `-Ofast` flag enables all `-O3` optimizations along with `-ffast-math`, which relaxes strict IEEE 754 floating-point compliance (e.g., permitting reciprocal approximations and ignoring NaN/signed zeros). Because CoreMark is strictly an **integer-only benchmark**, floating-point relaxations offer zero algorithmic benefit. The recorded throughput (**34,957.08 iter/s**) is practically equivalent to `-O3`, with slight variations attributable to background OS scheduling.

6. **Host Architecture Tuning via `-march=native`:** Specifying `-march=native` instructs LLVM to generate code tailored to the host Apple M2 instruction pipeline (leveraging ARMv8.5-A scheduling models). When applied alongside base `-O2`, it produces **34,655.81 iter/s** with the **lowest variance** ($\sigma = 157.83$) among all optimized runs, proving that microarchitecture-specific instruction alignment minimizes branch target buffer (BTB) conflicts and pipeline bubbles.
7. **CoreMark/MHz Microarchitectural Efficiency Analysis:** Normalizing throughput against clock frequency provides an architectural figure of merit:

$$\text{CoreMark/MHz} = \frac{\text{Mean Iterations/Sec}}{\text{Clock Frequency (MHz)}} = \frac{35376.00}{3492 \text{ MHz}} = \mathbf{10.13} \text{ CoreMark/MHz} \quad (1)$$

For comparison, embedded in-order RISC-V cores (such as RV32IMC) typically achieve **2.5–3.5 CoreMark/MHz**, while high-performance out-of-order cores reach **5.0–7.0 CoreMark/MHz**. The Apple M2 core's score of **10.13 CoreMark/MHz** underscores the exceptional instruction-level parallelism, massive out-of-order reorder buffer, and memory bandwidth of Apple Silicon.

4 Key Algorithms Implemented in CoreMark

To evaluate CPU cores without distortion from standard C library implementations, CoreMark self-contains four fundamental integer algorithms (detailed in the EEMBC CoreMark specification and *README.md*):

4.1 Linked List Processing (`core_list_join.c`)

- **Structure and Data Representation:** The algorithm utilizes a two-level indirect data structure:

```
typedef struct list_data_s { ee_s16 data16; ee_s16 idx; } list_data;
typedef struct list_head_s {
    struct list_head_s *next;
    struct list_data_s *info;
} list_head;
```

  Crucially, dynamic memory allocation (`malloc`) is avoided; the list resides in a pre-allocated static workspace. To model realistic heap fragmentation, **25% of the pointers are sequential in memory**, while **75% are non-sequential and scattered**.
- **Operations Executed:**
  1. *Find / Linear Traversal:* Executes sequential search operations seeking target values, traversing the pointer chain and feeding results into the output checksum.
  2. *Merge Sort by Data:* Executes an in-place merge sort based on `data16`, and computes a 16-bit CRC checksum across the sorted sequence.
  3. *Order Restoration Merge Sort:* Executes a second merge sort based on `idx`, restoring the list to its exact initial topological state for **algorithmic idempotence**.
- **Microarchitectural Stresses:** Exercises pointer chasing with two levels of indirection (`list_head` to `info` to `data16`), directly exposing L1 data cache miss latency, load-to-use pipeline stalls, and branch prediction during merge sort comparisons.

4.2 Matrix Manipulation (`core_matrix.c`)

- **Data Space Partitioning:** The 2000-byte workspace is partitioned equally across the three core algorithms (∼666 bytes each). For matrix manipulation, this slice allocates three $N \times N$ matrices ($N = 9$): input matrices A and B composed of 16-bit signed integers (`ee_s16`, 2 bytes/element) and result matrix C composed of 32-bit signed integers (`ee_s32`, 4 bytes/element), occupying exactly $81 \times (2 + 2 + 4) = 648$ bytes.
- **Computational Phases:**
  1. Multiplication of Matrix A by a runtime scalar constant k into Matrix C.
  2. Multiplication of Matrix A by a column vector from Matrix B into Matrix C.
  3. Full Matrix-Matrix Multiply ($A \times B \to C$) via triply nested loops ($C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}$).

  The upper bits of result elements are accumulated into the runtime CRC chain.
- **Microarchitectural Stresses:** Heavily stresses **integer multiplication throughput**, multiply-accumulate (MAC) pipeline depth, strided memory access patterns, and compiler loop unrolling.

4.3 Deterministic Moore State Machine (`core_state.c`)

- **Operation and Architecture:** A deterministic Moore state machine parses comma-separated input tokens classifying them into format categories: integers, floating-point representations, scientific notation, or invalid inputs. EEMBC explicitly selected a Moore model over a Mealy model because Mealy machines are table-driven (testing load/store operations), whereas Moore machines rely on nested conditional branches (`if-else`) and multi-way branch dispatch tables (`switch`).
- **Microarchitectural Stresses:** Stresses branch direction predictors, branch target buffers (BTB), and evaluates pipeline flush penalties resulting from mispredicted branches on pseudo-random token inputs.

4.4 CRC Validation and Anti-Optimization Barrier (`core_util.c`)

- **Dual Architectural Purpose:**
  1. *Algorithmic Verification:* Verifies mathematical correctness across all iterations using 16-bit CRC checksums.
  2. *Anti-Optimization Barrier:* Modern optimizing compilers eliminate dead code when results are unused. CoreMark prevents dead-code elimination by chaining intermediate results from all kernels into a volatile CRC accumulator, forcing full execution.

5 RISC-V Instruction Set Architecture (ISA) Extension Sets

In Lab 2, CoreMark will be deployed and profiled on the configurable **CORE-V Wally processor**. Wally is an open-source, highly parameterized 5-stage / multi-stage pipelined RISC-V CPU core. Understanding the target RISC-V ISA extensions is essential for anticipating performance trends across microarchitectural configurations:

5.1 RV32I: Base 32-bit Integer Instruction Set

- **Architectural Foundation:** The mandatory base RISC-V integer standard. It defines 32 general-purpose 32-bit registers (`x0`–`x31`, where `x0` is hardwired to zero), a 32-bit program counter, and fixed-width 32-bit instructions.
- **Instruction Set Repertoire (40 Base Instructions, 47 with Zicsr/Zifencei):**
  - R-Type (Register-Register ALU): `ADD`, `SUB`, `SLL`, `SLT`, `SLTU`, `XOR`, `SRL`, `SRA`, `OR`, `AND`.
  - I-Type (Immediate ALU, Loads, Jumps): `ADDI`, `SLTI`, `SLTIU`, `XORI`, `ORI`, `ANDI`, `SLLI`, `SRLI`, `SRAI`, `LB`, `LH`, `LW`, `LBU`, `LHU`, `JALR`.
  - S-Type (Stores): `SB`, `SH`, `SW`.
  - B-Type (Conditional Branches): `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`.
  - U-Type (Upper Immediates): `LUI`, `AUIPC`.
  - J-Type (Unconditional Jumps): `JAL`.
  - System / Environment: `ECALL`, `EBREAK`, `FENCE`. (Under the modern unprivileged RISC-V specification, CSR access instructions are modularized into the **Zicsr** extension and instruction synchronization into **Zifencei**, yielding 40 unprivileged base instructions or 47 when bundled with CSR/fence extensions).
- **Critical CoreMark Implication: RV32I possesses no hardware multiplier or divider.** All integer multiplications in `core_matrix.c` (matrix dot products, scalar scalings) must be emulated in software via compiler runtime routines (`__mulsi3`), which execute 30–40 shift-and-add instructions per multiply. This imposes a severe computational bottleneck on pure RV32I. (In contrast, CRC calculation in `core_util.c` relies solely on logical shifts, XOR, and bitwise masking, requiring no hardware multiplier).

5.2 RV32IM: Standard Extension for Integer Multiplication and Division

- **Hardware Enhancement:** Adds dedicated hardware multiplier and divider execution blocks in the ALU stage.
- **Added Instructions (8 Instructions):**
  - Multiplication: `MUL` (lower 32 bits), `MULH` (signed high 32 bits), `MULHU` (unsigned high 32 bits), `MULHSU` (signed-unsigned high 32 bits).
  - Division and Remainder: `DIV`, `DIVU`, `REM`, `REMU`.
- **CoreMark Impact:** Delivers the **single largest performance acceleration** in the benchmark suite. In `core_matrix.c`, the triple-nested loop performs N^3 multiplications. Replacing ∼35-cycle software emulation loops with 1- to 3-cycle pipelined hardware `MUL` instructions yields an anticipated 3× to 6× throughput improvement over RV32I.

5.3 RV32IMC: Standard Extension for Compressed Instructions

- **Architectural Foundation:** Adds 16-bit compressed instruction encodings that interleave freely with standard 32-bit instructions on any 16-bit aligned boundary (e.g., `C.LW`, `C.SW`, `C.ADD`, `C.J`, `C.BEQZ`).
- **Code Density:** Compresses compiled binary footprint by **25% to 35%**.
- **CoreMark Impact:** By significantly shrinking the static loop footprints, the C extension drastically improves **Instruction Cache (I-cache) hit rates**. On cache-constrained embedded processors such as Wally, fewer fetch cycles are wasted waiting for main memory or bus transfers, yielding substantial throughput gains.

5.4 RV32GC: General-Purpose ISA Standard

- **Architectural Definition:** The standard "G" bundle represents the full general-purpose platform:

$$\mathbf{G} = \mathbf{I} + \mathbf{M} + \mathbf{A} + \mathbf{F} + \mathbf{D} + \mathbf{Zicsr} + \mathbf{Zifencei} \tag{2}$$

  Combined with **C**, RV32GC represents the baseline hardware profile capable of hosting rich operating systems like Linux.
- **Constituent Extensions:**
  - **A (Atomic):** Adds load-reserved/store-conditional (`LR.W`/`SC.W`) and atomic memory operations (`AMO*`) for multi-core thread synchronization.
  - **F & D (Floating-Point):** Single- and double-precision IEEE 754 coprocessors with 32 dedicated floating-point registers (`f0`–`f31`) and the `fcsr` status register.
- **CoreMark Impact:** CoreMark's integer kernels do not invoke floating-point instructions; however, the 'A' extension enables multi-threaded CoreMark configurations (`-DMULTITHREAD=N`), permitting linear performance scaling across multiple cores.

5.5 RV64I: Base 64-bit Integer Instruction Set

- **Architectural Foundation:** Expands the integer register file and datapath to **XLEN = 64 bits**.
- **Instruction Set Additions:** Adds 64-bit load/store instructions (`LD`, `SD`) and word-specific instructions operating on 32-bit sub-words with sign-extension (e.g., `ADDIW`, `ADDW`, `SUBW`, `SLLW`, `SRLW`, `SRAW`, `LWU`).
- **CoreMark Trade-Offs:** While 64-bit registers enable single-cycle 64-bit arithmetic, pointers double in size from 4 bytes to 8 bytes. In `core_list_join.c`, this increases cache line footprint and L1D cache pressure compared to RV32.

6 Comparative Synthesis and Projections for CORE-V Wally (Lab 2)

6.1 Architectural Comparison Matrix

Table 2 synthesizes the architectural characteristics and expected CoreMark performance profiles across the evaluated RISC-V ISA configurations.

Table 2: Comprehensive RISC-V ISA Architectural Comparison for CORE-V Wally

Architectural Metric	RV32I	RV32IM	RV32IMC	RV32GC	RV64I
Register File Width (XLEN)	32 bits	32 bits	32 bits	32 bits (Int) + 64 bits (FP)	64 bits
Hardware Multiplier/Divider	None (Emulated)	Dedicated Pipelined Unit	Dedicated Pipelined Unit	Dedicated Pipelined Unit	None (Emulated)
Compressed Encoding Support	No (32-bit only)	No (32-bit only)	Yes (16/32-bit mixed)	Yes (16/32-bit mixed)	No (32-bit only)
Floating-Point Coprocessor	None	None	None	Single (F) & Double (D)	None
Relative Code Size	1.00× (Baseline)	≈ 0.98×	≈ **0.70**×	≈ 0.72×	≈ 1.05 × –1.10× (64-bit Immediates)
Expected CoreMark Relative to RV32I	1.00× (Baseline)	**3.5** × –**5.5**×	**4.0** × –**6.2**×	4.0 × –6.2× (Single-Core)	0.95 × –1.10×
Primary Pipeline Bottleneck	Software `__mulsi3`	Memory / Branch stalls	Instruction fetch stalls	FPU silicon unutilized	L1D pressure (8B pointers)

6.2 Empirical Hypotheses for Lab 2 Simulation

Based on our host profiling and architectural analysis, we formulate the following testable hypotheses for Lab 2 on CORE-V Wally:

1. **Hypothesis 1 (The Hardware Multiplier Acceleration):** Upgrading from RV32I to RV32IM will yield a massive jump ($> 3.5\times$) in CoreMark throughput. In `core_matrix.c`, the triple-nested loops execute tens of thousands of multiplications. Hardware `MUL` execution eliminates thousands of software emulation instructions.
2. **Hypothesis 2 (Instruction Cache Benefits of Compressed Instructions):** Enabling the C extension (RV32IMC) will reduce binary size by $\sim$30%, significantly increasing instruction cache hit rates in Wally's tightly constrained I-cache. This will translate to a measurable 10%–20% performance boost over RV32IM.
3. **Hypothesis 3 (Neutral Impact of F and D in Single-Threaded CoreMark):** Profiling RV32GC against RV32IMC will reveal nearly identical single-thread CoreMark throughput because CoreMark contains no floating-point kernels. However, RV32GC incurs greater silicon area overhead.
4. **Hypothesis 4 (Pointer Overhead in RV64I):** Comparing RV64I to RV32I will demonstrate that 64-bit architectures without hardware multiplication suffer slightly higher data cache pressure due to 8-byte pointer storage in `core_list_join.c`.

7 Conclusion

This prelab investigation established a rigorous empirical and architectural baseline for processor benchmarking:

1. **Compiler Optimization Impact:** On the host Apple M2, compiler optimization levels produce a **4.17**× throughput expansion (from 8,474.61 iter/s at `-O0` to 35,376.00 iter/s at `-O3`). The primary gain (4.03×) occurs at `-O1` through SSA-based register allocation, with incremental gains at `-O3` driven by loop unrolling.
2. **Microarchitectural Efficiency:** Normalizing host performance against the 3.49 GHz clock frequency yields an outstanding score of **10.13** CoreMark/MHz, reflecting the high instruction-level parallelism of Apple Silicon.
3. **Algorithmic Diversity:** CoreMark's four kernels comprehensively stress key CPU subsystems: pointer chasing and data cache latency (linked list), ALU multiplication and register pressure (matrix), branch prediction and pipeline flush penalties (Moore state machine), and algorithmic execution integrity (CRC).
4. **Preparedness for Lab 2:** The theoretical foundation established in this report directly equips us to profile, evaluate, and interpret architectural trade-offs across RV32I, RV32IM, RV32IMC, RV32GC, and RV64I on the CORE-V Wally processor.